PANDAS

Class Quick Reference

Beginner-friendly notes • Python data analysis

This guide reorganizes the topics and examples from your uploaded Pandas class notebook. It keeps the same learning scope: loading data, understanding DataFrames and Series, creating DataFrames, selecting and changing data, iteration, sorting, and filtering.

Before you start: Install Pandas with `pip install pandas`. Some file formats need an extra reader package: Excel commonly uses `openpyxl`; Parquet commonly uses `pyarrow` or `fastparquet`.

Contents

1. 1. Pandas, Series, and DataFrame
2. 2. Loading data from files
3. 3. Basic DataFrame inspection
4. 4. Creating DataFrames from Python data
5. 5. Accessing rows and columns: loc and iloc
6. 6. Changing the index and column names
7. 7. Updating and deleting data
8. 8. Iterating through a DataFrame
9. 9. Sorting data
10. 10. Filtering data with conditions
11. 11. Quick revision and common mistakes

Example dataset used in this guide

The original notebook reads `student_data.csv`, but that CSV was not included with the uploaded notebook. To make the selection, update, sorting, and filtering examples runnable on their own, the following small sample DataFrame uses the same column names found in your class notebook.

```
import pandas as pd

df = pd.DataFrame({
    "StudentID": ["PH1001", "PH1002", "PH1003", "PH1004", "PH1005"],
    "FullName": ["Alif Rahman", "Fatima Akhter", "Imran Hossain",
                 "Jannatul Ferdous", "Kamal Uddin"],
    "Data Structure Marks": [85, 92, 88, 78, 72],
    "Algorithm Marks": [85, 92, 88, 78, 72],
    "Python Marks": [88, 90, 84, 82, 76],
    "CompletionStatus": ["Completed", "Completed", "Not Started",
                         "Completed", "Not Started"],
    "EnrollmentDate": ["2024-01-10", "2024-01-20", "2024-02-10",
                       "2024-02-12", "2024-03-05"],
    "Instructor": ["Mr. Karim", "Ms. Salma", "Mr. Karim",
                   "Ms. Salma", "Mr. Karim"],
    "Location": ["Dhaka", "Chattogram", "Dhaka", "Sylhet", "Chattogram"]
})
```

1. Pandas, Series, and DataFrame

What is Pandas?

Pandas is a Python library used to work with structured data. It is commonly used to read, inspect, select, modify, sort, and filter tabular data.

Example

```
import pandas as pd
```

DataFrame

A DataFrame is a two-dimensional table with rows and columns. It is similar to a spreadsheet or database table.

Syntax

```
df = pd.read_csv('student_data.csv')
```

Example

```
print(type(df))
```

Expected result <class 'pandas.core.frame.DataFrame'>

Series

A Series is a one-dimensional labeled array. Selecting one column from a DataFrame normally returns a Series.

Syntax

```
series = df['StudentID']
```

Example

```
print(type(df['StudentID']))
```

Expected result <class 'pandas.core.series.Series'>

Important: A DataFrame is 2D; a Series is 1D. Selecting one column with one pair of square brackets returns a Series.

2. Loading data from files

Pandas provides reader functions for several common file formats. The file path must point to a real file that your Python session can access.

CSV files

CSV (comma-separated values) is a common text format for tabular data.

Syntax

```
df = pd.read_csv('student_data.csv')
```

Example

```
csv_data = pd.read_csv('student_data.csv')
print(csv_data)
print(type(csv_data))
```

Important: The notebook uses `student_data.csv`. Keep the file in the notebook's working directory, or provide its full path.

Excel files

Use `read_excel()` to load an Excel workbook, such as an `.xlsx` file.

Syntax

```
excel_file = pd.read_excel('phitron_student_marks.xlsx')
```

Example

```
excel_file = pd.read_excel('phitron_student_marks.xlsx')
print(excel_file)
print(type(excel_file))
```

Important: If Pandas reports that an Excel engine is missing, install `openpyxl` with `pip install openpyxl`.

Parquet files

Parquet is a column-oriented data file format often used for efficient storage and analysis.

Syntax

```
parquet_file = pd.read_parquet('students.parquet')
```

Example

```
parquet_file = pd.read_parquet('students.parquet')
print(parquet_file)
print(type(parquet_file))
```

Important: A Parquet engine is required. A common option is `pip install pyarrow`.

JSON files

JSON stores data using objects and arrays. `read_json()` loads supported JSON structures into Pandas.

Syntax

```
json_file = pd.read_json('data.json')
```

Example

```
json_file = pd.read_json('data.json')
print(json_file)
print(type(json_file))
```

Important: The exact JSON structure affects how Pandas interprets the data. The notebook's `data.json` file is not included, so its structure cannot be confirmed here.

3. Basic DataFrame inspection

head() and tail()

`head()` displays the first rows; `tail()` displays the last rows. The default is five rows.

Syntax

```
df.head()          # first 5 rows
df.head(4)       # first 4 rows
df.tail()          # last 5 rows
df.tail(3)       # last 3 rows
```

Example

```
print(df.head(4))
print(df.tail(2))
```

Important: The notebook's comment says `head()` and `tail()` show a chosen number of rows. Pass the number in parentheses; without it, Pandas uses five.

columns

`df.columns` returns the column labels. Converting it to a NumPy array is possible.

Syntax

```
df.columns
```

Example

```
import numpy as np

col = np.array(df.columns)
print(col)
print(col.dtype)
```

Expected result The column labels are usually stored as object/string-like values in the NumPy array.

index

`df.index` describes the row labels. The default index created by Pandas is usually a range of integers.

Syntax

```
df.index
```

Example

```
ind = np.array(df.index)
print(ind)
print(ind.dtype)
```

Expected result For a default index with five rows: [0 1 2 3 4], with an integer dtype.

info()

`info()` prints a concise summary: row count, column names, non-null counts, data types, and memory usage.

Syntax

```
df.info()
```

Important: Use `info()` to quickly check whether columns contain missing values. A non-null count smaller than the row count indicates missing data.

sample()

`sample()` returns random rows from the DataFrame.

Syntax

```
df.sample(3)
```

Important: The notebook repeats `df.sample(10)` twice. The repeated cell does not introduce a new method. If the DataFrame has fewer than 10 rows, `sample(10)` raises an error unless sampling with replacement.

describe()

`describe()` calculates summary statistics for numeric columns by default, including count, mean, standard deviation, minimum, quartiles, and maximum.

Syntax

```
df.describe()
```

Important: It does not calculate statistics for every column by default. Text columns are generally excluded from the default numeric summary.

DataFrame dtypes

A DataFrame has a `dtypes` attribute (plural), which reports the data type of each column.

Syntax

```
df.dtypes
```

Important: Use `df.dtypes`, not `df.dtype`. A DataFrame does not have a single `dtype` because different columns can have different data types.

4. Creating DataFrames from Python data

The notebook demonstrates creating a DataFrame from a list, tuple, dictionary, and list of dictionaries.

From a list of lists

Each inner list becomes one row. Use `columns` to provide column names and `index` to set row labels.

Syntax

```
pd.DataFrame(data, columns=[...], index=[...])
```

Example

```
my_list = [['Alice', 25], ['Bob', 30], ['Charlie', 28]]

list_df = pd.DataFrame(
    my_list,
    columns=['Name', 'Age'],
    index=[1, 2, 3]
)
print(list_df)
```

From a tuple of tuples

A tuple can also be used as the row data. If you do not provide an index, Pandas creates the default integer index.

Syntax

```
tuple_df = pd.DataFrame(my_tuple, columns=['Name', 'Age'])
```

Example

```
my_tuple = (('Alice', 25), ('Bob', 30), ('Charlie', 28))

tuple_df = pd.DataFrame(my_tuple, columns=['Name', 'Age'])
print(tuple_df)
```

From a dictionary of lists

Dictionary keys become column names, and the values in each list become column values. All column lists must have equal lengths.

Syntax

```
dict_df = pd.DataFrame(my_dict)
```

Example

```
my_dict = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 29, 28]
}

dict_df = pd.DataFrame(my_dict)
print(dict_df)
```

From a list of dictionaries

Each dictionary represents a row. If a key is missing from one row, Pandas fills that cell with a missing value (`NaN`).

Syntax

```
data_df = pd.DataFrame(my_data)
```

Example

```
my_data = [
    {'Name': 'Alice', 'Age': 25, 'City': 'New York'},
    {'Name': 'Bob', 'Age': 30, 'City': 'Paris'},
    {'Name': 'Charlie', 'Age': 28}
]

data_df = pd.DataFrame(my_data)
print(data_df)
```

Remember: In `data_df`, Charlie's City is missing, so Pandas displays `NaN`. Selecting `data_df['Name']` returns a Series.

```
print(type(data_df['Name']))
```

5. Accessing rows and columns: loc and iloc

Use square brackets for simple column selection. Use `.loc` for label-based selection and `.iloc` for integer-position-based selection.

Selecting one column

One column name inside square brackets returns a Series.

Syntax

```
df['FullName']
```

Example

```
names = df['FullName']
print(names)
print(type(names))
```

loc: select by labels

`.loc` selects rows and columns using their labels. Its general form is `.loc[row_labels, column_labels]`.

Syntax

```
df.loc[row_selection, column_selection]
```

Example

```
first_row = df.loc[0]
selected_rows = df.loc[[1, 3]]
selected_range = df.loc[1:3]

one_column = df.loc[:, 'Python Marks']
multiple_columns = df.loc[:, ['Python Marks', 'Algorithm Marks']]
row_and_column = df.loc[1:3, 'CompletionStatus']
```

Important: The ending label in a `.loc` slice is included. `df.loc[1:3]` includes labels 1, 2, and 3 when those labels exist. The colon `:` means all rows or all columns, depending on its position.

iloc: select by integer position

`.iloc` selects rows and columns by their zero-based integer positions, not by index labels.

Syntax

```
df.iloc[row_positions, column_positions]
```

Example

```
first_row = df.iloc[0]
first_three_rows = df.iloc[0:3]
first_five_columns = df.iloc[:, 0:5]
```

Important: Unlike `.loc`, the stop position in `.iloc` is excluded. Therefore `0:3` selects positions 0, 1, and 2.

Common confusion: After `df.set_index('StudentID')`, the row labels become student IDs. A call such as `df_index.loc[0]` will not find the old row label 0. Use a real StudentID with `.loc`, or use `.iloc[0]` for the first row by position.

6. Changing the index and column names

set_index()

Use `set_index()` to make an existing column the DataFrame index. By default, it returns a new DataFrame and leaves the original unchanged.

Syntax

```
df_index = df.set_index('StudentID')
```

Example

```
df_index = df.set_index('StudentID')
print(df_index.head())
```

Important: To modify the original DataFrame instead, use `df.set_index('StudentID', inplace=True)`. Do not assign the result to a new variable and also expect the original `df` to change.

rename()

Use `rename(columns=...)` to change selected column names. The dictionary maps old names to new names.

Syntax

```
df.rename(columns={'old_name': 'new_name'}, inplace=True)
```

Example

```
df.rename(
    columns={
        'FullName': 'Full Name',
        'Algorithm Marks': 'Algo Marks'
    },
    inplace=True
)
print(df.columns)
```

Important: The notebook renames `FullName` to `Full Name` and `Algorithm Marks` to `Algo Marks`. Because `inplace=True` is used, the existing DataFrame is changed.

7. Updating and deleting data

Updating a cell with loc

Use `.loc[row_label, column_label]` to assign a new value to one cell.

Syntax

```
df.loc[row_label, column_label] = new_value
```

Example

```
df.loc[1, 'Python Marks'] = 90
df.loc[1, 'CompletionStatus'] = 'Completed'
```

Important: The row label must exist. If you changed the index, use the new index labels.

Updating several rows

A label slice can be used to update a range of rows in a column.

Syntax

```
df.loc[1:3, 'Python Marks'] += 2
```

Example

```
df.loc[1:3, 'Python Marks'] += 2
```

Important: `.loc[1:3]` includes both endpoint labels. This operation adds 2 to the selected values; missing values remain missing.

Deleting rows with drop()

By default, `drop()` removes rows whose index labels are supplied.

Syntax

```
df.drop(0, inplace=True)
```

Example

```
df.drop(0, inplace=True)
```

Important: This removes the row whose index label is 0; it does not mean 'remove the first row' in every possible index. Use `df.drop(index=0)` for clarity.

Deleting columns with drop()

Use `axis=1` or the `columns` argument to remove a column.

Syntax

```
df.drop('Instructor', axis=1, inplace=True)
```

Example

```
df.drop(columns=['Instructor'], inplace=True)
```

Important: `axis=1` means columns; `axis=0` means rows. The `columns=` form is often easier to read. `inplace=True` changes the existing DataFrame.

8. Iterating through a DataFrame

iterrows()

`iterrows()` yields each row as a pair: the row's index label and a Series containing that row's values.

Syntax

```
for index, row in df.iterrows():
    ...
```

Example

```
for index, row in df.iterrows():
    print(f'{index}:\n{row}')
```

Important: The variable named `series` in the notebook is simply the row Series; the name is not special. Iterating row by row can be slower than using Pandas operations for large data.

itertuples()

`itertuples()` yields each row as a named tuple by default. With `index=False`, the index is omitted.

Syntax

```
for row in df.itertuples(index=False):
    ...
```

Example

```
for row in df.itertuples(index=False):
    print(row)
```

Important: The notebook uses `index=False`, so each tuple contains the row's column values but not its index.

9. Sorting data

sort_values(): ascending order

`sort_values()` returns a new DataFrame sorted by the selected column. By default, the order is ascending.

Syntax

```
sorted_df = df.sort_values('Data Structure Marks')
```

Example

```
sorted_df = df.sort_values('Data Structure Marks')
print(sorted_df)
```

Important: The original DataFrame is not changed unless you use `inplace=True` or assign the result back to `df`.

Descending order

Set `ascending=False` to sort from largest to smallest.

Syntax

```
sorted_df = df.sort_values('Data Structure Marks', ascending=False)
```

Example

```
sorted_df = df.sort_values('Data Structure Marks', ascending=False)
```

Sorting by multiple columns

Pass a list of column names to sort by more than one column. Pandas uses the next column to break ties in the previous column.

Syntax

```
sorted_df = df.sort_values(
    ['Data Structure Marks', 'Python Marks'],
    ascending=[False, True]
)
```

Example

```
sorted_df = df.sort_values(
    ['Data Structure Marks', 'Python Marks'],
    ascending=[False, True]
)
```

Important: The notebook's `ascending=False` applies descending order to both columns. Its `ascending=[0, 1]` is valid because `0` means False and `1` means True, but `[False, True]` is clearer: Data Structure Marks descending, then Python Marks ascending.

10. Filtering data with conditions

Filtering returns only the rows that satisfy a Boolean condition. In the notebook, the condition is passed inside `.loc[...]`.

Filter by one condition

Use `==` to compare a column with a value. The result is a Boolean Series that can be used to select matching rows.

Syntax

```
result = df.loc[df['CompletionStatus'] == 'Not Started']
```

Example

```
not_started = df.loc[df['CompletionStatus'] == 'Not Started']
completed = df.loc[df['CompletionStatus'] == 'Completed']
```

Filter with two conditions (AND)

Use `&` when both conditions must be true. Put each condition inside parentheses.

Syntax

```
result = df.loc[(condition_1) & (condition_2)]
```

Example

```
completed_ds90 = df.loc[
    (df['CompletionStatus'] == 'Completed') &
    (df['Data Structure Marks'] >= 90)
]
print(completed_ds90)
```

Important: Use `&` for element-by-element AND in Pandas conditions, not the Python keyword `and`. Each comparison must be wrapped in parentheses.
Other useful comparison symbols: `==` equal to | `!=` not equal to | `>` greater than | `<` less than | `>=` greater than or equal to | `<=` less than or equal to

11. Quick revision and common mistakes

Task	Pandas code
Read a CSV	pd.read_csv('file.csv')
First / last rows	df.head() / df.tail()
Column labels	df.columns
Row index	df.index
Summary / data types	df.info() / df.dtypes
Numeric statistics	df.describe()
Select one column	df['ColumnName']
Select by labels	df.loc[rows, columns]
Select by positions	df.iloc[row_positions, col_positions]
Set index	df.set_index('ColumnName')
Rename columns	df.rename(columns={'old': 'new'})
Delete rows / columns	df.drop(index=0) / df.drop(columns=['Name'])
Iterate rows	df.iterrows() / df.itertuples()
Sort	df.sort_values('ColumnName', ascending=False)
Filter	df.loc[df['ColumnName'] == value]

Common mistakes to avoid

- **`df.dtype` —** Use `df.dtypes` for the data type of each DataFrame column.
- **Confusing loc and iloc —** `.loc` uses labels and includes the stop label in slices; `.iloc` uses positions and excludes the stop position.
- **Using `and` in a Pandas filter —** Use `&` between conditions and wrap each comparison in parentheses.
- **Forgetting `inplace=True` —** Methods such as `drop()` and `rename()` usually return a new DataFrame unless you assign the result or use `inplace=True`.
- **Assuming set_index changes the original —** `df.set_index(...)` returns a new DataFrame by default.
- **Missing input files —** A read function cannot load a file that is absent or located somewhere else.

Source coverage and limitations

All non-empty topic cells in the uploaded notebook were reviewed. The guide includes: importing Pandas; Series and DataFrame; CSV, Excel, Parquet, and JSON readers; DataFrame inspection; creating DataFrames from lists, tuples, and dictionaries; column/row selection; `.loc` and `.iloc`; setting an index; renaming columns; updating and deleting data; row iteration; sorting; and condition-based filtering.

The notebook refers to these external data files: `student_data.csv`, `phitron_student_marks.xlsx`, `students.parquet`, and `data.json`. They were not included with the uploaded `.ipynb`, so their complete contents and the exact JSON structure could not be checked. The guide retains the original file-loading examples and provides a small in-memory student DataFrame so the main practice examples can run without those files.

Corrections made: clarified that a DataFrame uses `dtypes` rather than `dtype`; corrected and explained the distinction between label-based `.loc` and position-based `.iloc`; clarified inclusive `.loc` slices; explained that `set_index()` returns a new DataFrame unless used with `inplace=True`; made the multi-column sort directions explicit; and improved deletion syntax with the `index=` and `columns=` arguments. The repeated `sample(10)` cell was consolidated because it did not add a new topic.